RTL Design Sherpa

mem-ctrl-ip Family Overarching Hardware Architecture Specification 0.1

October 10, 2026

Table of Contents

1 Common Has Index....7

2 mem-ctrl-ip Common IP — Micro-Architecture Specification....8

2.1 Document Organization....8

2.1.1 Front Matter....8

2.1.2 Chapter 1: The Family Package....8

2.1.3 Chapter 2: mc_axi4_layer and the Host-Side FUBs....8

2.1.4 Chapter 3: mc_storage_layer and the Entry-Pool FUBs....8

2.1.5 Chapter 4: mc_scheduler_layer and the Pick Machinery....8

2.1.6 Chapter 5: mc_training_layer....8

2.1.7 Chapter 6: mc_ras_layer....9

2.1.8 Chapter 7: mc_dfi_2p1_layer and the Shared DFI Pipeline....9

2.1.9 Chapter 8: mc_dfi_3p1_layer....9

2.1.10 Chapter 9: mc_dfi_4p0_layer....9

3 Document Information....10

3.1 Revision history....10

3.2 Related documents....11

4 Purpose, Scope, and the Two Lines....12

4.1 Purpose....12

4.2 The premise: one common core, many controllers....12

4.3 The two lines: research and product....12

4.4 What this book binds....13

4.5 Conventions....13

5 The Architecture: axi4 → storage → scheduler → (ras) → training → dfi.....14
5.1 The layer contract.....14
5.2 Why layers — the reason this shape exists.....16
5.3 The three structural rules of the stack.....16
5.4 What the rocks own.....16
6 The Descriptor Seam.....18
6.1 The contract.....18
6.2 The invariants.....18
6.3 What the seam is not.....19
6.4 Where this stands in the rocks.....19
7 Why dfi_2p1_layer / dfi_3p1_layer / dfi_4p0_layer Are Separate Blocks.....21
7.1 The reasoning.....21
7.2 What is shared.....21
7.3 What the layer never does.....22
8 Doctrines Added by the Common-IP Campaign.....23
8.1 6. Timing CSRs are cycle counts — frequency coupling is explicit.....23
8.2 7. Parameterize width, never personality.....23
8.3 8. Safety is structure, policy is a CSR, and the two never mix.....23
8.4 9. RAS features are discussed as needed — never enumerated ahead.....24
8.5 10. The return path is its own contract — and perfect is the enemy of good 24
8.6 11. A block with one customer stays rock-side.....24
9 The common-ip Layout.....25
9.1 The blocks (as of 2026-10-10).....25

9.2 The ownership rule (restated so this book is self-sufficient)....26
9.3 Documentation map....26
10 Products and Research: granite / gabbro / diorite....28
10.1 The product names....28
10.2 What a product may leave out — and what it may not....28
10.3 The research line's role....29
10.4 Relationship to the rocks' books....29

List of Figures

No figures in this document.

List of Tables

No tables in this document.

1 Common Has Index

Generated: 2026-10-11

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 mem-ctrl-ip Common IP — Micro-Architecture Specification

Version: 0.1 **Date:** 2026-10-10 **Status:** v0.1. One chapter per common block. The architecture these blocks implement — the layer stack, the descriptor seam, the doctrines — is the family HAS (common_has); this book is the per-block microarchitecture and never restates the architecture. Where the two disagree, the HAS wins and this book is the defect.

> **Read this first.** Every claim in a chapter is one of: (a) read from the RTL it documents (cited `file:line`), (b) inherited from a named rock's measured history (cited to the rock's book or bug record), or (c) an explicitly marked open question. Nothing else is asserted. Silicon-bug history (`BUG-*`, `ISSUE-*`, measured regression numbers) is invariant documentation and is carried verbatim from the RTL comments.

2.1 Document Organization

Note: All chapters linked below for automated document generation.

2.1.1 Front Matter

- Document Information

2.1.2 Chapter 1: The Family Package

- `mc_common_pkg` — types, memtype enum, shared opcodes

2.1.3 Chapter 2: `mc_axi4_layer` and the Host-Side FUBs

- `mc_axi4_layer` — AXI slaves, intakes, coalescing, return ring

2.1.4 Chapter 3: `mc_storage_layer` and the Entry-Pool FUBs

- `mc_storage_layer` — CAMs, read snooping, bank image

2.1.5 Chapter 4: `mc_scheduler_layer` and the Pick Machinery

- `mc_scheduler_layer` — schedulable-entry view, timers, arbiter

2.1.6 Chapter 5: `mc_training_layer`

- `mc_training_layer` — shell, maintenance slot, rock wrappers

2.1.7 Chapter 6: `mc_ras_layer`

- `mc_ras_layer` — skeleton, transform pair, reserved CSR surface

2.1.8 Chapter 7: `mc_dfi_2p1_layer` and the Shared DFI Pipeline

- `mc_dfi_2p1_layer` — DFI 2.1 command/data pipeline + CDC

2.1.9 Chapter 8: `mc_dfi_3p1_layer`

- `mc_dfi_3p1_layer` — the 3.1 generation delta

2.1.10 Chapter 9: `mc_dfi_4p0_layer`

- `mc_dfi_4p0_layer` — the 4.0 generation delta

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Document Information

Field	Value
Document	mem-ctrl-ip Family — Overarching Hardware Architecture Specification
Version	0.1
Date	2026-10-10
Status	Partner-review draft — the owner reads and comments before scoria/andesite books are moved to this stack
Owner	mem-ctrl-ip family (common-ip/docs/)
Audience	Anyone building, porting, or productizing a family memory controller
Binding force	Family level: controller books reference this; they never restate it

3.1 Revision history

Version	Date	Change
0.1	2026-10-10	First edition. The Phase-2 common-IP campaign (2026-10-09 → 2026-10-10) extracted the shared package, eight layers, and seventeen FUBs into common-ip; this book records the architecture those blocks implement — the partner's layered-stack vision — as the family's binding contract. Doctrines added by the campaign are Chapter 4; the

Version	Date	Change
		original five doctrines remain in 02_family_doctrine.md and are not restated.

3.2 Related documents

Document	Relationship
common_mas	the per-block microarchitecture of every block this book names
02_family_doctrine.md	the original five doctrines; Chapter 4 adds to them, never replaces them
01_mem_ctrl_pkg.md	the shared package design (memtype enum, shared types)
03_dfi_boundary_lineage.md	the DFI 2.1 → 3.1 → 4.0 boundary story this book's Chapter 3 summarizes
Rock HAS/MAS books (pumice, scoria, andesite)	per-rock instantiations of this architecture; deltas recorded there, not here

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

4 Purpose, Scope, and the Two Lines

4.1 Purpose

This book is the architecture contract for the mem-ctrl-ip family. It exists so that a person can answer, from one document:

1. What the layers are, what each one owns, and how they fit together.
2. What the interface between the layers is — the seam — and which properties of it are invariant.
3. Which rules (doctrines) every block and every rock follows without re-litigating them.
4. What "productizing" a research controller means in this family, and which block serves which product.

The book deliberately does **not** contain: per-block microarchitecture (that is common_mas), register maps (rock books), JEDEC or DFI clause-level detail (04_jedec_generation_deltas.md, the simplified-spec books), or verification plans (rock books). A book that answers every question answers none; the ownership rule keeps each question at one address.

4.2 The premise: one common core, many controllers

Three research controllers exist — pumice (DDR2/LPDDR2), scoria (DDR3/LPDDR3), andesite (DDR4/LPDDR4) — and for a long time each carried its own copy of the same logic: the AXI intakes, the CAMs, the timers, the arbiter, the DFI pipeline. The Phase-2 campaign (2026-10-09 → 2026-10-10) extracted what had two live customers into common-ip: one package (`mc_common_pkg`), eight layers, seventeen FUBs. The premise of this book is that the extraction is not a code-dedup exercise — it is an architecture. The blocks in common-ip are the family; the rocks are instantiations.

The test for what moves to common-ip is the **two-customer rule** (from the common-ip README, predating the campaign): a block moves here only when it has two live customers at extraction time. Speculative one-customer abstractions stay in the research controllers. This rule is why common-ip contains what it contains and — just as important — why the rocks still own their init sequencers, mode registers, refresh controllers, and page-policy engines: those blocks are generation-specific today, and a common core with one customer would be a guess.

4.3 The two lines: research and product

The family runs two lines off the same common core:

- **Research** — pumice, scoria, andesite. Full-featured controllers that prove out generations, run the characterisation harnesses, and close timing on real boards. They are where features land first and where measured numbers come from.
- **Product** — granite (v2: DDR2/LPDDR2), gabbro (v3: DDR3), diorite (v4: DDR4/LPDDR4). Realistic, area-conscious SKUs built from the same layers. Product naming and the product line's reason to exist are Chapter 5.2.

granite doubles as the **bypass-validation testbed**: because the v2 generation needs no storage-layer reordering and no RAS features, a granite build with the storage layer in its lite configuration and the RAS layer bypassed is the cheapest full proof that the common core works without its optional layers (Ch 5.2).

4.4 What this book binds

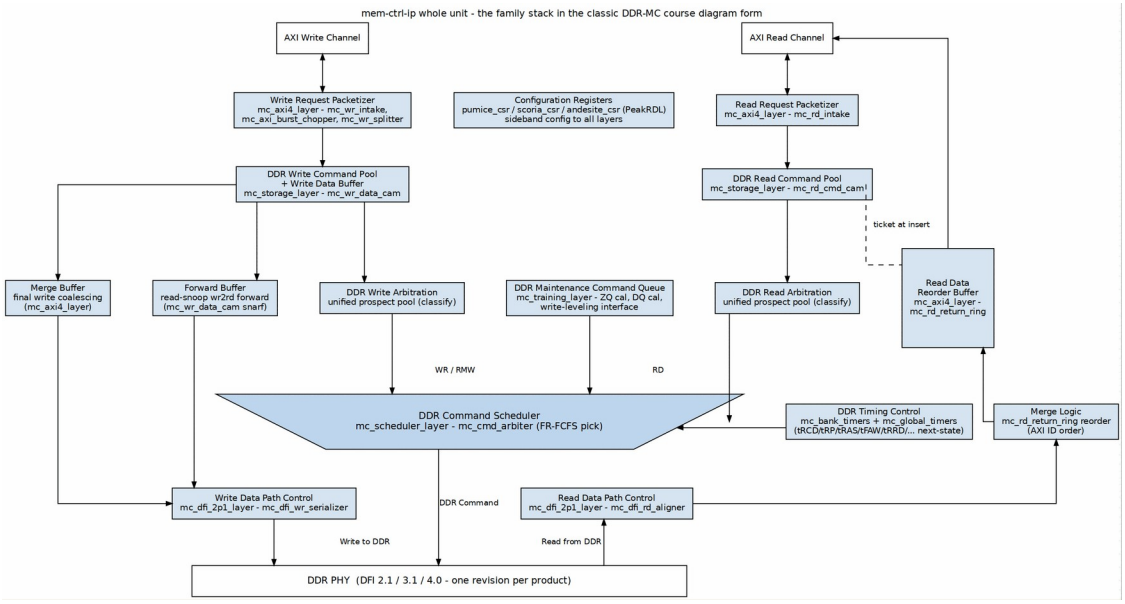
- The layer stack and the name and role of every layer (Ch 2).
- The descriptor seam and its invariants (Ch 2.2).
- The DFI layer naming — `dfi_2p1_layer`, `dfi_3p1_layer`, `dfi_4p0_layer` (Ch 3).
- Doctrines added by the campaign (Ch 4). The original five doctrines in 02_family_doctrine.md remain in force; this book never restates them, it adds to them.
- The common-ip layout and the product/research line-up (Ch 5).

4.5 Conventions

The family's conventions are inherited, not invented here: markings are relative and match everywhere (INHERITED / MODIFIED / NEW); the evidentiary rule (inherited, cited, or open question — nothing else); registers are named, not offset-numbered; bandwidth figures carry the theoretical maximum beside them. Where this book needs a new convention it says so at the point of use.

5 The Architecture: axi4 → storage → scheduler → (ras) → training → dfi

This chapter is the vision, written down. It was set out by the partner in the Phase-2 campaign and it drives every block in common-ip. The layers below are listed in command-flow order. The whole unit, drawn in the family's diagram house style (STYLE.md — the classic DDR-MC course form the owner fixed as the reference):



The mem-ctrl-ip whole unit — AXI channels at top, the layer stack in the middle, DDR PHY bar at the bottom

5.1 The layer contract

Each layer has exactly one job. The table is the binding statement; the per-block microarchitecture is in common_mas and is not restated here.

Layer	Job	Owns	Does not own
`mc_axi4_layer`	Turn host AXI4 traffic into memory-command descriptors	AXI4 slave intakes, shallow FIFOs, burst chop/split, final coalescing; read-data return endpoint	DRAM knowledge of any kind; scheduling; the DRAM data path

Layer	Job	Owns	Does not own
`mc_storage_layer`	Hold the schedulable-entry pool and the bank image	rd/wr CAMs (or SRAM-backed equivalents), read snooping, per-bank open-row image as a *select*, address mapping	The pick itself; DRAM command encoding
`mc_scheduler_layer`	Pick the next DRAM command	The schedulable-entry view contract, bank/global timers, command arbiter, maintenance arbitration (refresh, ZQ, training windows)	Where entries are stored (that is the storage layer's job); PHY timing
`mc_ras_layer` (optional)	Reliability transforms on the data path	ECC insert/check as a transform pair, scrub as a maintenance client	Anything when not instantiated — its absence is a true bypass, not a stub
`mc_training_layer`	The training control plane as a maintenance client	ZQ calibration control, DQ calibration, write-leveling interface, the maintenance slot arbitration that carries them	The training *search* (PHY responsibility, per doctrine — the family doc's boundary rule)
`mc_dfi_NpM_layer`	Deliver commands and data to the PHY at a DFI revision	Command formatter/pipeline, write serializer, read aligner, the aclk↔dfi_clk CDC — per generation:	Scheduling decisions; training searches

Layer	Job	Owns	Does not own
		`dfi_2p1_layer`, `dfi_3p1_layer`, `dfi_4p0_layer`	

5.2 Why layers — the reason this shape exists

The point of the stack is **drop-in performance features on a stable contract**. Because the storage layer presents the scheduler the same descriptor view the AXI layer does, a feature that needs a different entry-pool organization (bigger CAM, SRAM-backed pool, hashed bank assignment) changes the storage layer only — the scheduler and everything downstream do not move. A product that needs no reordering replaces the storage layer with its lite configuration and keeps the same scheduler (Ch 5.2). The seam is the architecture; the blocks are replaceable parts on it.

This is also why the scheduler is defined against a **schedulable-entry view** rather than against the CAMs directly: the pick machinery argmaxes over schedulable-entry vectors, and *something* has to hold those entries — that something is the storage layer in a full build, and the stripped AXI layer serving N=1 in a lite build. “Storage off” can never be literally zero hardware (the entry store cannot reach zero); it is the lite config, not an omission.

5.3 The three structural rules of the stack

1. **Command flow is one direction.** Descriptors flow axi4 → storage → scheduler → dfi. Nothing upstream is consulted after a pick commits; the seam’s advisory snapshot is the boundary (Ch 2.2).
2. **Maintenance requests; it never preempts** (family doctrine 2, in force here unchanged). Refresh, ZQ, calibration are guests on the command bus with a request/grant contract into the scheduler’s maintenance slots.
3. **Optional layers are param-selected, not compile-selected by absence.** The RAS layer exists or it does not; a build without it wires the storage and DFI layers together directly. What is *never* optional: the entry store, the write-data buffer (it cannot reach zero), and the CDC.

5.4 What the rocks own

Generation-specific blocks stay rock-side today, by the two-customer rule: the init sequencer, the mode register, the refresh controller, the page policy engine, and the power-down controller. The scheduler layer is a thin rock wrapper around the common scheduler core precisely for this: the wrapper instantiates the rock’s children and maps the rock’s maintenance source into the generic maintenance slot (mc_scheduler_knobs.md §6 documents the union that made this possible).

When a second customer appears for one of those blocks — the refresh controller family is the leading candidate — the extraction follows the same rule that produced this stack.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 The Descriptor Seam

The seam is the interface between layers, and the load-bearing idea of this architecture: **the storage layer presents the scheduler exactly the same descriptor interface the AXI layer does.** One shape, two providers. This chapter states the contract and its invariants.

6.1 The contract

A layer upstream of the scheduler exposes a pool of schedulable entries:

- **N entry slots** (`N = 8` in the full storage configuration; `N = 1` in the stripped AXI-layer configuration — "argmax set to 1 for both reads and writes, then it matches to a FIFO"). The full configuration is not a different protocol; it is the same vectors with more entries behind them.
- **Per-entry vectors:** the scheduler's pick machinery argmaxes over schedulable-entry vectors — for each entry, its validity, read/write class, decoded address (row / bank / column), age, and readiness inputs. The vectors are the contract; the storage behind them (CAM, SRAM, a FIFO in the N=1 case) is an implementation freedom of the providing layer.
- **Commit / issue removal:** when the scheduler fires a command built from an entry, it removes the entry (or the consumed portion of it) through the same interface. Partial consumption — a chopped burst issuing over multiple cycles — is the provider's bookkeeping, invisible downstream.

Because the AXI layer can be designed to emit the same information for a single in-order item, a controller can be assembled with no storage layer at all and the scheduler cannot tell the difference from the vectors alone. That is what makes the lite product SKU (Ch 5.2) a configuration rather than a redesign.

6.2 The invariants

These hold at every seam in the family. They are recorded here so a performance-feature drop-in cannot break one silently.

1. **Read snooping sees un-coalesced writes.** The read path snoops the write stream at write-data-CAM granularity, before any write coalescing. A merged descriptor would hide write data a later read must see; so snooping is defined against the un-coalesced view and coalescing happens after.
2. **Merged descriptors inherit the oldest age.** When the storage layer coalesces entries, the result carries the age of its oldest constituent. Age

drives the scheduler's starvation avoidance; inheriting anything newer would let a coalesced burst jump the queue.

3. **The advisory snapshot / epoch is the boundary.** What the scheduler sees is an advisory snapshot of the pool, refreshed at an epoch boundary. The scheduler does not reach back upstream after a pick commits; the epoch is the consistency point. Anything stronger (live querying of the pool) couples the layers in a way the drop-in goal forbids.

6.3 What the seam is not

- **The return path is not on the seam.** Read data flows storage → AXI through `mc_rd_return_ring` — a separate contract with its own ordering rules (it must reorder to AXI IDs). It cannot ride the descriptor seam, and the symmetry goal must not absorb it: *never let the perfect get in the way of the good.* A future revision may unify the seams; until then the descriptor seam is command-direction only.
- **The open-row image is not a seam payload.** The per-bank open-row image stays scheduler-adjacent (the storage layer provides it as a *select* feeding the scheduler's own view). It does not cross the seam as an independent interface; that would let two layers believe two different things about which row is open.
- **The seam is not a bus.** No arbitration crosses it; the upstream layer owns its entries until the scheduler consumes them.

6.4 Where this stands in the rocks

Today's rocks wire the seam concretely as `mc_axi4_layer → mc_storage_layer → <rock>_scheduler_layer`, with the CAMs living in the storage layer and the rock scheduler wrapper presenting the rock's maintenance clients to the common core. pumice is the reference integration (measured at 66.67 / 75 MHz on Nexys A7). The contract in this chapter is the destination the storage-layer extraction already implements; when a rock's documentation and this contract disagree, this contract wins and the rock book is the defect.

7 Why dfi_2p1_layer / dfi_3p1_layer / dfi_4p0_layer Are Separate Blocks

The DFI boundary layers are named for the DFI revision they implement — `mc_dfi_2p1_layer`, `mc_dfi_3p1_layer`, `mc_dfi_4p0_layer` — and the naming is deliberate: **the DFI revision is a family boundary, not a parameter.** This chapter says why, and what is shared underneath.

7.1 The reasoning

A controller's DFI revision is decided by the memory generation it serves (DDR2/LPDDR2 → DFI 2.1, DDR3/LPDDR3 → DFI 3.1, DDR4/LPDDR4 → DFI 4.0) and by the PHY it must talk to. It is not something a single bitstream flips at runtime, and — per the parameter-sustainability doctrine (Ch 4) — a parameter that changes protocol generation fails the test: the legal combinations explode, and the DV cost is per-generation anyway. So each revision is its own named block, a product picks exactly one, and the product line maps one-to-one: granite → `dfi_2p1_layer`, gabbro → `dfi_3p1_layer`, diorite → `dfi_4p0_layer`.

The parameter-sustainability rule applies *within* a generation: widths, phase counts, FIFO depths are parameters; the protocol generation is not.

7.2 What is shared

The blocks share the pipeline that is revision-independent, extracted once as common FUBs and instantiated by whichever generation layer needs them:

Shared FUB	Role
`mc_dfi_cdc`	the single `aclk` ↔ `dfi_clk` crossing (async FIFOs + init handshake)
`mc_dfi_cmd_path`	command pipeline from scheduler to formatter
`mc_dfi_wr_serializer`	write-data packing/serialization to the DFI bus
`mc_dfi_rd_aligner`	read-data capture/alignment from the DFI bus

The revision-specific part is the command encoding and control surface: the 2.1 formatter, the 3.1 command path, the 4.0 command path (ACT_n, gear-down, CA parity/alert_n, DBI). Those are the per-generation FUBs (`mc_dfi_3p1_cmd_path`, `mc_dfi_4p0_cmd_path`) — and the reason the layer exists at all is that the *combination* of shared pipeline plus generation command surface is what a product instantiates.

The full clause-level lineage is 03_dfi_boundary_lineage.md and the simplified-spec books; the per-layer microarchitecture is common_mas ch7–ch9. This chapter fixes only the naming and the family-boundary rule.

7.3 What the layer never does

Per the family boundary doctrine, the DFI layer never absorbs PHY responsibilities: no training search, no calibration algorithm, no IO timing closure. The training layer speaks *to* the DFI layer through the documented CSR/contract surface (phy_cal_csr_contract.md); the PHY owns the mechanism. A DFI layer that starts calibrating is the boundary doctrine's defect case, and the review that catches it is this chapter's reason for existing.

8 Doctrines Added by the Common-IP Campaign

The original five doctrines live in 02_family_doctrine.md and remain in force; they are not restated here. The Phase-2 campaign (2026-10-09 → 2026-10-10) settled six more, recorded as doctrine 6 onward in the same shape: one rule, its reason, and where it was learned.

8.1 6. Timing CSRs are cycle counts — frequency coupling is explicit

Every timing CSR in the family holds a count of MC-clock cycles, never nanoseconds. Its correct value is a function of the MC clock frequency; a frequency change invalidates the whole timing set at once, and software must re-derive and reprogram it before traffic resumes. Stale values violate real DRAM timings and present as data corruption, not errors.

Reason. pumice's own books carried timing registers "in cycles" without ever saying what binds them to; the LiteDRAM `BUILD_CLK_HZ` reading 100 MHz on a 75 MHz engine clock (2026-10-03 char results) is the same failure operating in a shipping flow. Learned 2026-10-10; every HAS now carries the warning at the register map, and this doctrine makes it family-wide.

8.2 7. Parameterize width, never personality

Build parameters express widths, depths, and counts — never behavioral strategy. Runtime-selectable behavior is a CSR/config bit (doctrine 1); strategy selection that would require illegal parameter cross-products is a family boundary, not a parameter. The test, three parts: (a) all parameter combinations must be legal; (b) a parameter changes the amount of elaborated hardware, not runtime behavior; (c) DV cost stays linear in the parameter, not per-product.

Reason. The refresh-controller divergences across the three rocks ({REFpb, Mode A/B/C, FGR, MPC, ZQCS}) fail (a) — the cross-products are illegal — so generation features of refresh stay in thin rock children over an extractable `mc_refresh_base`, per the two-customer rule. Page-policy strategies pass (runtime `policy_mode` enum) but will split at ~4+ strategies. Learned from the scheduler-merge knob inventory (mc_scheduler_knobs.md, 2026-10-10).

8.3 8. Safety is structure, policy is a CSR, and the two never mix

Safety-critical behavior is fixed RTL structure — no runtime bit may disable it. Runtime-selectable policy (page policy, refresh mode, placement) is a CSR. When a feature could be either, decide first which it is; if a runtime bit could violate a DRAM or DFI safety limit, the bit is the defect.

Reason. The campaign's review of "which refresh features go behind params" produced exactly three mechanisms — CSRs for policy, params/family boundary for features, structure for safety — and the rule exists so the question is asked once, at design time, not per pull request.

8.4 9. RAS features are discussed as needed — never enumerated ahead

The RAS layer (`mc_ras_layer`) is defined as a transform pair (ECC insert/check) plus scrub-as-maintenance-client, with `HAS_ECC` ∈ {OFF, DETECT, CORRECT} and a reserved CSR surface. Beyond that, RAS features are added only when a customer needs them, at which point the partner decides. The skeleton is deliberately *not* a menu.

Reason. “Possible more features in the ras layer are controlled by params. we will discuss as needed.” (partner, 2026-10-10.) A layer whose feature list is speculative is a layer nobody can validate; the transform pair is the only RAS structure with two customers today.

8.5 10. The return path is its own contract — and perfect is the enemy of good

Read data returns on `mc_rd_return_ring` under its own ordering contract (AXI-ID reorder); it does not ride the descriptor seam, and the seam’s symmetry goal must not absorb it. Unify the seams in a future revision if a real customer appears; until then a separate, honest contract beats an elegant one that is wrong.

Reason. The descriptor seam (Ch 2.2) is command-direction; forcing the read-data reorder through it would couple every drop-in feature to AXI ordering. Partner ruling, 2026-10-10: “never let the perfect get in the way of the good.”

8.6 11. A block with one customer stays rock-side

The two-customer rule, promoted from the common-ip README to doctrine: a block moves to common-ip only when it has two live customers at extraction time; speculative one-customer abstractions stay in the research controllers no matter how clean the dedup looks.

Reason. Common-ip exists so features compose across controllers, not so that code is written once. Every block in common-ip today met the rule at extraction; every block still rock-side (init sequencer, mode register, refresh personalities, page policy, power-down) is there because it has one customer, and its extraction is a later campaign’s decision, made when the second customer exists.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

9 The common-ip Layout

common-ip holds what no single controller owns. The layout is the architecture of Chapter 2 made into a directory tree.

```
common-ip/
├── rtl/
│   ├── includes/mc_common_pkg.sv      the family package: memtype
enum,
│   │                                  shared types, opcode encodings
│   ├── fub/                           the seventeen shared FUBs
│   ├── macro/                         the eight layers
│   └── filelists/                     .f files: one per FUB/layer +
mc_common_all.f
├── dv/                                shared verification collateral
(grows per campaign)
└── docs/                              family docs (this book,
doctrine, lineage,
                                       knobs inventories, per-block
MAS)
```

9.1 The blocks (as of 2026-10-10)

Package. `mc_common_pkg` — the family memtype enum (six members, one LP bit + two-bit generation field; design owned by 01_mem_ctrl_pkg.md), shared timing-struct inventory, shared opcodes.

Layers (macro/).

Layer	State	Customers
`mc_axi4_layer`	landed	pumice (measured); scoria/andesite scheduled
`mc_storage_layer`	landed	pumice; scoria/andesite scheduled
`mc_scheduler_layer`	landed (merged from three rock schedulers, −3,465 lines)	all three rocks adopt via thin wrappers
`mc_training_layer`	shell + rock wrappers	pumice carries ZQ/DQ cal behind it
`mc_ras_layer`	skeleton per doctrine 9 — instantiated nowhere	none yet; that is honest

Layer	State	Customers
mc_dfi_2p1_layer	landed	pumice (measured on Nexys A7)
mc_dfi_3p1_layer	landed	scoria
mc_dfi_4p0_layer	landed	andesite

Shared FUBs (fub/). mc_addr_mapper, mc_axi_burst_chopper, mc_bank_timer, mc_bank_timers, mc_cmd_arbiter, mc_dfi_cdc, mc_dfi_cmd_path, mc_dfi_wr_serializer, mc_dfi_rd_aligner, mc_dfi_3p1_cmd_path, mc_dfi_4p0_cmd_path, mc_global_timers, mc_rd_cmd_cam, mc_rd_intake, mc_rd_return_ring, mc_wr_data_cam, mc_wr_intake, mc_wr_splitter. Each is owned by a layer and documented in that layer's common_mas chapter; the knobs inventories (mc_*_knobs.md beside this book) record the parameter surface per block.

9.2 The ownership rule (restated so this book is self-sufficient)

A design or doctrine that no single controller owns lives at this level; a controller's book references these documents and never restates them. A block enters common-ip through the two-customer rule (doctrine 11). A block leaves only by obsolescence of all its customers.

9.3 Documentation map

Doc	Owns
this book (common_has/)	the layer-stack architecture, the seam, DFI naming, new doctrines
common_mas/	per-block microarchitecture of every block above
01_mem_ctrl_pkg.md	the shared package design
02_family_doctrine.md	the original five doctrines
03_dfi_boundary_lineage.md	DFI clause-level lineage
04_jedec_generation_deltas.md	JEDEC generation reuse argument
mc_*_knobs.md	parameter/knob inventories (measured, dated)
phy_cal_csr_contract.md	the training↔PHY contract surface

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

10 Products and Research: granite / gabbro / diorite

The family runs two lines off one common core (Ch 1). This chapter names the product line and states what a product is allowed to leave out — the "realistic version" of each memory controller, as distinct from the research controllers that prove the generations out.

10.1 The product names

Product	Generation	Memory	DFI layer
granite	v2	DDR2 / LPDDR2	`mc_dfi_2p1_layer`
gabbro	v3	DDR3 / LPDDR3	`mc_dfi_3p1_layer`
diorite	v4	DDR4 / LPDDR4	`mc_dfi_4p0_layer`

The naming is igneous, ordered, and owned here: future generations extend the list (basalt is reserved informally for the v5/DDR5 generation per the memtype encoding's 2'b11 code). A product is a configuration of the common stack — layers selected, knobs set, rock children chosen — not a new RTL tree.

10.2 What a product may leave out — and what it may not

The v2 generation needs no bank groups, no FGR, no DBI, no CA parity. A granite build correspondingly runs:

- **Storage layer in its lite configuration.** "Off" is not literal: the scheduler's pick machinery argmaxes over schedulable-entry vectors, and something must hold those entries. Lite means the stripped AXI-layer configuration serving N=1, in-order — the seam vectors with one entry behind them (Ch 2.2). The entry store can shrink toward one; it cannot reach zero.
- **RAS layer absent.** Unlike storage, this is a true bypass: no transform pair instantiated, no CSR surface beyond the reserved decode, scrub absent. Doctrine 9 governs what may later appear here.
- **The write-data buffer at its minimum.** It cannot reach zero — write data must be staged somewhere between the seam and the serializer — but a product sets it by the SKU's burst profile, not by the research characterisation harness's sweep needs.

What a product may **never** leave out: the entry store, the write-data staging, the aclk↔dfi_clk CDC, and the maintenance request/grant path (doctrine 2 — a product that cannot refresh correctly is not a smaller product, it is a broken one).

granite is therefore the **bypass-validation testbed**: the cheapest full proof that the common core runs correctly with its optional layers at minimum and absent, before gabbro/diorite spend area on features the v2 generation will never see.

10.3 The research line's role

pumice, scoria, andesite keep their jobs: generations land there first, measured numbers come from there, and a feature's second customer comes from there. The product line consumes the proven stack; it does not pioneer on it. When a product needs a feature the research line has not proven (a product-specific ODT profile, an industrial temperature derate), the feature lands in the research line first, by doctrine 11's logic even when the product is the only paying customer — because the research harnesses are where a number can be produced to explain it.

10.4 Relationship to the rocks' books

The rock HAS/MAS books record each research controller as it stands, with deltas against the architecture of this book marked INHERITED / MODIFIED / NEW. A product book, when the first product is built, is a short document: this book, the chosen configuration, and the product's own board integration. Nothing more — a product whose book is long is a product whose configuration is not yet decided.